○ E-LEARNING · STUDY NOTES

Nicloud · Kubernetes on AWS

Module 1 – Helm and EKS Add-ons

Contents

Learning objectives 2
Architecture 2
Helm and charts 2
Wrapper chart 3
Example: the Karpenter wrapper 3
Subchart values 3
Example: high availability and IRSA 4
IRSA: AWS permissions without keys 4
Example: scheduled backups with Velero 4
Manifests after the chart 5
Summary 5
Quiz 5
Additional resources 6

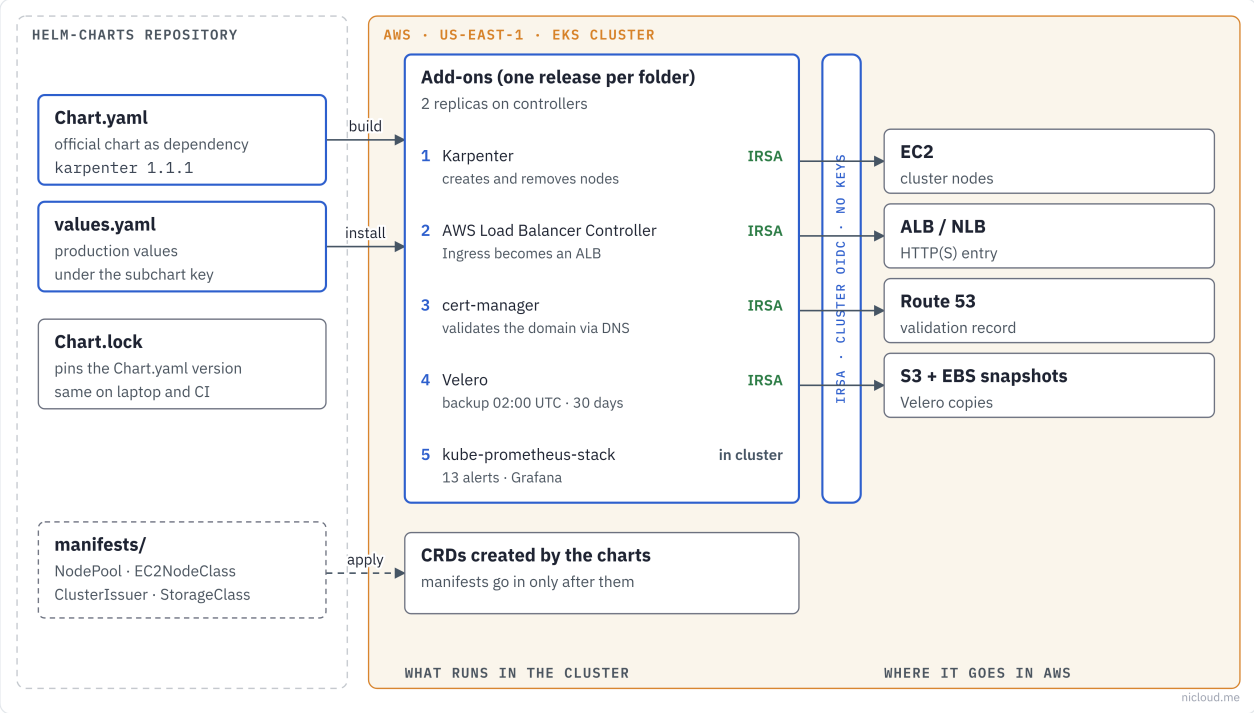
Learning objectives

In this module, you will learn how to:

- Describe what a Helm **chart** is and what it packages.
- Explain the **wrapper chart** pattern and why it makes add-on upgrades easier.
- Use `helm dependency build` and understand the role of **Chart.lock**.
- Override **subchart** values in `values.yaml`.
- Give a pod AWS permissions with **IRSA**, with no stored keys.
- Configure high availability and scheduled backups for add-ons.
- Explain why some manifests can only be applied after the chart.

Architecture

Overview of what this repository does, in the Nicloud diagram style: callers on the left, rules in the middle, destinations on the right.



Helm and charts

Helm is the package manager for Kubernetes. A **chart** is the package: a `Chart.yaml` (name, version, dependencies), a `values.yaml` (the values you can change) and the templates that become manifests in the cluster.

Installing a chart creates a **release**. Releases keep history, so you can upgrade (`helm upgrade`) and go back (`helm rollback`) with one command.

When to use

Use Helm for third-party software you will upgrade many times (Karpenter, cert-manager, Prometheus). For a single manifest of your own, `kubectl apply` or Kustomize is enough.

Wrapper chart

This repository does not copy the official charts. Each folder is a **wrapper chart**: a small chart that declares the official chart as a **dependency** and only stores production values.

The benefit is that upgrading an add-on becomes changing a version number. The templates stay the maintainer's.

Example: the Karpenter wrapper

```
# karpenter/Chart.yaml
apiVersion: v2
name: karpenter
version: 0.1.0
appVersion: "1.1.1"
dependencies:
  - name: karpenter
    version: 1.1.1
    repository: "oci://public.ecr.aws/karpenter"
```

How it works

(1) Fetch the dependency

`helm dependency build` reads `Chart.yaml`, downloads the official chart into `charts/` and honors the versions recorded in **Chart.lock**. The lock guarantees your machine and CI install exactly the same version.

(2) Install with production values

`helm install karpenter . -n kube-system -f values.yaml` installs the wrapper, which in turn installs the official chart with the values defined here.

Subchart values

The official chart's values go **under a key named after the dependency**. Without that key, the value goes to the wrapper and the official chart never sees it.

Example: high availability and IRSA

```
# cert-manager/values.yaml
cert-manager:
  crds:
    enabled: true
  replicaCount: 2
  serviceAccount:
    annotations:
      eks.amazonaws.com/role-arn:
"arn:aws:iam::ACCOUNT_ID:role/CERT_MANAGER_ROLE_NAME"
```

- `replicaCount: 2` keeps the add-on running if a node goes down.
- `crds.enabled: true` installs the **CRDs** (the new types, such as `Certificate` and `ClusterIssuer`) together with the chart.

IRSA: AWS permissions without keys

IRSA (IAM Roles for Service Accounts) links a pod's **service account** to an **IAM role**. The pod receives temporary credentials through the cluster's OIDC provider, with no access key in a Secret.

The `eks.amazonaws.com/role-arn` annotation is the whole link on the Kubernetes side. On the AWS side, the role must trust the cluster's OIDC provider and that specific service account.

Who uses IRSA in this repo

Add-on	Why it needs AWS
Karpenter	create and remove EC2 instances
AWS Load Balancer Controller	create ALBs and NLBs
cert-manager	create Route 53 records to validate certificates
Velero	write backups to S3 and create EBS snapshots

Example: scheduled backups with Velero

```
# velero/values.yaml
velero:
  schedules:
    daily-backup:
      schedule: "0 2 * * *"
      template:
        ttl: "720h"
```

The backup runs every day at 02:00 UTC and each copy lives for **720 hours (30 days)**. The restore runbook is in `disaster-recovery.md`.

Manifests after the chart

Some folders have `manifests/` to apply **after** the chart, such as Karpenter's `NodePool` and `EC2NodeClass` and cert-manager's `ClusterIssuer`. Order matters: these resources use types that only exist once the chart installs the **CRDs**. Applying them first fails with an unknown kind error.

Summary

Helm packages Kubernetes software into **charts**, and each install becomes a **release** with history that you can upgrade and roll back.

This repository uses **wrapper charts**: each add-on declares the official chart as a dependency and only keeps production values. Upgrading means changing the version in `Chart.yaml`, and **Chart.lock** guarantees the same version everywhere. The official chart's values live under the key named after the dependency.

Add-ons that talk to AWS use **IRSA**: the service account points to an IAM role and the pod gets temporary credentials, with no stored keys. Production values run **two replicas** for high availability, and **Velero** takes a daily backup kept for 30 days.

Finally, resources that depend on **CRDs** (NodePool, ClusterIssuer) are applied only after the chart.

Quiz

You need to upgrade cert-manager to the new version. What changes in the repository?

- The dependency version in `Chart.yaml` and `Chart.lock` (with `helm dependency update`). The templates are the maintainer's.

You set `replicaCount: 2` at the root of the wrapper's `values.yaml` and cert-manager still runs one replica. Why?

- The value must go under the `cert-manager:` key, which is the dependency name.

Velero needs to write to S3. What is the recommended way to grant that permission on EKS?

- IRSA: annotate the service account with the IAM role. No access keys in Secrets.

`kubectl apply -f karpenter/manifests/default-nodepool.yaml` failed with "no matches for kind NodePool". What was missing?

- Installing the Karpenter chart first. It creates the `NodePool` CRD.

What is `Chart.lock` for?

- Pinning the exact dependency versions, so CI and your machine install the same thing.

The Velero backup runs at 02:00 UTC with `ttl: 720h`. How many days is each backup kept?

- 30 days.

Additional resources

- Charts (Helm docs)
- helm dependency
- Values best practices
- IAM roles for service accounts (EKS)
- Karpenter
- AWS Load Balancer Controller
- Velero